



UiT The Arctic University of Norway

Project 4

Multiple Processes

Mike Murphy

UiT INF-2203

Spring 2026

A close-up photograph of a laptop screen and keyboard. The screen displays a terminal window with a message that reads "t... Seems Ok. Now go get some sleep :).". The keyboard is visible below the screen, with a red dot highlighting the 'H' key.

t... Seems Ok. Now go get some sleep :).

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

P4 Multiple Processes

- ▶ P1: Boot and run a program
- ▶ P2: Protect against misbehaving processes
- ▶ P3: Multithreading
- ▶ **P4: Multiple Processes**

What we're going to do:

- ▶ Shell as a process in user space
- ▶ Support multiple processes with multiple address spaces
- ▶ Launch other processes
- ▶ Wait for other processes (or let them run in background)
- ▶ Inter-Process Communication (IPC)
 - ▶ Implement pipes between processes (Maybe. Precode not ready.)
 - ▶ Implement message passing between processes (Maybe. Precode not ready.)
- ▶ Implement more sync abstractions:
 - ▶ monitors + condition variables: bounded buffer problem
 - ▶ barriers: checkpoints

Notes: P4 2026

- ▶ Precode is extra incomplete / buggy
 - ▶ Code is more open-ended, less scaffolding
 - ▶ Fewer hints in code
- ▶ We need to be in dialog for this assignment
 - ▶ Please ask clarifying questions to TAs and on Discord
 - ▶ I will be watching Discord and answering questions
- ▶ Time waits for no one!
 - ▶ Deadline is more firm due to administrative deadlines
 - ▶ Very little wiggle room

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

Unix Pipes

- ▶ The output of one program is the input to another
- ▶ Example: How many .txt files are in this directory?

```
ls | grep ".txt" | wc -l
```

1. `ls` (list) files, one per line
2. `grep` (filter) print only lines that contain `.txt`
3. `wc -l` (word count, lines) count lines

- ▶ Key part of the **Unix Philosophy**: small programs that work together:
 - ▶ “Write programs that do one thing and do it well.”
 - ▶ “Write programs to work together.”
 - ▶ “Write programs to handle text streams, because that is a universal interface.”¹

¹Philosophy summarized by Peter H. Salus, attributed to Douglas McIlroy.

More Shell, Less Egg

Problem: Find most frequently used words in some sample text

Six-line solution by Douglas McIlroy²³:

```
tr -cs A-Za-z '\n' |  
tr A-Z a-z |  
sort |  
uniq -c |  
sort -rn |  
sed ${1}q
```

1. "Make one-word lines by transliterating the complement (-c) of the alphabet into newlines (note the quoted newline), and squeezing out (-s) multiple newlines."
2. "Transliterate upper case to lower case."
3. "Sort to bring identical words together."
4. "Replace each run of duplicate words with a single representative and include a count (-c)."
5. "Sort in reverse (-r) numeric (-n) order."
6. "Pass through a stream editor; quit (q) after printing the number of lines designated by the script's first parameter (\${1})."

²Original paper: **Programming pearls: a literate program**. Bently et al, 1989.

³Blog post with summary and commentary: **More Shell, Less Egg**. Dr. Drang. 2001. 

More Classic Unix Utilities

- ▶ `cat` (concatenate): simply print input / concatenate multiple inputs
- ▶ `more/less` (pagers): scroll input interactively one page at a time
- ▶ `head/tail`: print the first/last n lines of input
- ▶ `cut`: treat each line as record with fields, print only certain fields
- ▶ `sed` (stream editor): filter and transform text

To learn more:

- ▶ `man <program>` — manual pages
 - ▶ `man 1 <program>`
 - ▶ `man 2 <syscall>`
 - ▶ `man 3 <library fn>`
- ▶ `apropos <search term>` — search man pages

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

Address Translation

TODO

Address Spaces

TODO

Address Space Switching

TODO

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

Unix: Fork/Exec

TODO

Munix Custom Syscall: Process Spawn

TODO

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

Monitors and Condition Variables

TODO

Barriers

TODO

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

1. Implement Address Space Setup with Allocation

Task:

- ▶ Current process load simply sets memory position
- ▶ Instead, use page allocation for each user-space page

Tests:

- ▶ Processes still load without wild memory errors
- ▶ Processes still exit cleanly
- ▶ No out-of-memory error after many process runs (no leaking pages)

1a. Address Space Hints

- ▶ Current `process_load_path` calls `addrspc_map` to set permissions for each segment it loads
 - ▶ Create an `addrspc_map_alloc` that takes same parameters, but allocates physical pages to map
 - ▶ For each page of segment, allocate (`physpage_alloc`) and map (original `addrspc_map`)
- ▶ Don't forget to allocate a page for user stack
- ▶ Don't forget to free pages (`physpage_free`) on cleanup

2. Implement process-spawn syscall

Task:

- ▶ Implement `process_spawn` syscall to launch new process
- ▶ This is a custom simpler alternative to `fork/exec`

Test Process: `ushell.c`

- ▶ Run the userspace shell (`ushell.c`) and use it to launch processes
- ▶ Processes should also launch and exit cleanly

2a. Process-Spawn Hints

- ▶ Run `ushe11` under Linux, run some commands, and compare
- ▶ Pay attention to separate address spaces
 - ▶ Don't forget to switch address spaces during `thread_switch`
 - ▶ Watch out for address space switch in process load
 - ▶ Try writing an address-space-switch function with log trace
- ▶ Be careful when passing arguments
 - ▶ If the `argv` pointers point to strings in `ushe11` address space,
 - ▶ and loading and starting new process switches to new address space,
 - ▶ those pointers point to same addresses, but different data.
 - ▶ See new helper fn in precode: `copy_argv`

3. Implement waitpid syscall

Task:

- ▶ Implement `waitpid` syscall (`wait4`-style for compatibility with Linux)

Tests:

- ▶ Run a process like `plane` in `ushe11`. It should wait until `plane` is done to show prompt again.
- ▶ Try `plane & hello` vs `plane; hello`
 - ▶ `plane & hello` — see `plane` fly and `hello` message at same time
 - ▶ `plane; hello` — `hello` message does not appear until after `plane`

Hints:

- ▶ This is a lot like `thread-join`
- ▶ On process exit, don't free the process struct until it is done being waited on

4. Implement condition variables

Task:

- ▶ Implement condition variable APIs in `lib/mulibc/threads.[ch]`
- ▶ Based on C11 API

Test Process: `bounded.c`

- ▶ Threads send and receive messages
- ▶ Messages/data should not be lost

Hints:

- ▶ Like before with mutexes, a simple spinning wait is ok
- ▶ You can also yield when spinning to give up the time slice faster
- ▶ You can do a simple `gcc` compile to run with GNU's condition variables

Compile on host with glibc

```
gcc -g -o ./bounded src/process/bounded.c  
./bounded`
```

5. Implement barriers

Task:

- ▶ Implement barrier APIs in `lib/mulibc/threads.[ch]`
- ▶ This is a custom API for Unix

Test Process: `footrace.c`

- ▶ Racers should not lap each other

Hints:

- ▶ You can implement a barrier using a condition variable
- ▶ Simple `gcc` compile will not work, since barrier API is custom to unix

6. Discuss Pipes

In your report, discuss the concept of Unix pipes

- ▶ What are pipes?
- ▶ Why are they useful?
- ▶ How would you adjust the process-spawn process to implement them?
- ▶ What additional system calls would we need to implement in order to implement pipes and make them useful?

Optional Challenge: Try to implement pipes

- ▶ If you are feeling ambitious, try to actually implement pipes.
- ▶ I will work on some support precode this week. Watch for updates.

Intro

Overview

Background

Command Line and Pipes

Multiple Processes, Multiple Address Spaces

Spawning New Processes

More Synchronization

Project 4

Project 4 Tasks

Admin

Two Ways to Continue

1. **Continue** building on your solution

- ▶ Git branch `main`
- ▶ Starts with P1 precode, adds only new precode (no solutions!)
- ▶ Should (*should!*) merge cleanly with your projects: `git merge main`
- ▶ More challenging, but more rewarding. OS is more *yours*
- ▶ More chances for bugs. Will have to do more troubleshooting

2. **Start fresh** with common project base

- ▶ Git branches: `p2-pre`, `p3-pre`, `p4-pre`
 - ▶ Starts with my solution to last project, then adds new precode
 - ▶ Check out, do not merge: `git checkout p2-pre` (or `p3-pre`, `p4-pre`, etc.)
 - ▶ Use common project base, focus only on current project
 - ▶ A more straightforward project experience
- ▶ Remember: This is new code. **There will be bugs either way!**

Hand-In Format: Same as Last Time

- ▶ Design reviews mid-way
 - ▶ Meet with TA
 - ▶ Convince them you have started and have a plan for solving this
- ▶ Hand in: Code and Report
 - ▶ Zip up code tree + git repo, upload to Canvas
 - ▶ Be sure to clean your code tree: `make clean`
 - ▶ DO NOT include cross compiler or bootloader
 - ▶ DO NOT include `out/` directory
 - ▶ DO include `.git/` directory
 - ▶ Zip should be a handful of Megabytes
 - ▶ Cross compiler will balloon it to Gigabytes (no thanks!)
 - ▶ Report
 - ▶ See [Report Guidelines doc](#) for detail
 - ▶ Upload report PDF to Canvas separately from code Zip

Extra Material

Project 4 Files List

More Info and Links

P4: New in precode

new	pre	task	file	desc
!	369		process/ushell.c	Shell as process
!	303		process/bounded.c	Bounded buffer (condvar test)
!	213		process/footrace.c	ASCII race (barrier test)

P4: Updated in precode

User side:

new	pre	task	file	desc
	90	48	lib/mulibc/threads.[ch]	Condvar and barrier APIs
	74		lib/mulibc/stdio.[ch]	I/O read API
	57		lib/mulibc/unistd.[ch]	read/spawn/wait syscall APIs
	3		lib/sys/syscall.h	read/spawn/wait syscall numbers

Kernel side:

new	pre	task	file	desc
	1	4	kernel/syscall_dispatch.c	Wire up I/O read syscall
	48	111	kernel/process.[ch]	Implement I/O read syscall
	1	77	kernel/pagemap.[ch]	Increase max phys pages
	5		kernel/interrupt.c	Print active page map on fault

P4: Task files

new	pre	task	file	desc
	1	77	kernel/pagemap.[ch]	Implement map w/ allocation
	48	111	kernel/process.[ch]	Update process handling
	1	4	kernel/syscall_dispatch.c	Wire up spawn/wait syscalls
	90	48	lib/mulibc/threads.[ch]	Implement condvars/barriers

Estimated project workload: 240 lines

Extra Material

Project 4 Files List

More Info and Links

Useful Resources I

- ▶ Textbook: *Modern Operating Systems* 4th ed.
 - ▶ Section 2.3.7: Monitors (and condition variables)
 - ▶ Section 2.3.8: Message Passing (bounded buffer problem)
 - ▶ Section 2.3.9: Barriers
- ▶ Threads:
 - ▶ [C11 <thread.h> API](#):
Reference for C11 threading and mutex APIs, which we mimic in our code
 - ▶ Programming with Threads ([PDF in repo](#))
A tutorial on threads in general